

Evaluación Sumativa 01 — Métricas de Proceso, Proyecto y Producto de Software

Modalidad: **Individual**

Formato de entrega: **PDF único** que incluya texto, grafo (como imagen embebida) y código refactorizado.

Contexto

Eres el líder técnico de **ShopFast**, una startup de e-commerce. El equipo de 5 desarrolladores trabaja con Scrum (sprints de 2 semanas) y TDD. El proyecto tiene 12 módulos en C#. Llevan 4 sprints y presentan estos problemas:

- **Retrasos:** 2 de 4 sprints no cumplieron la meta.
 - **Bug en producción:** un error crítico en el módulo de pagos llegó a producción.
 - **Código difícil de modificar:** agregar una nueva promoción tomó 3 días en vez de 1 estimado.
 - **Cobertura de pruebas:** está en una "zona desconocida".
-

Parte 1 — Métricas en dominios de proceso y proyecto (40 %)

Se te entregan estos datos del proyecto:

Sprint	Story Points planificados	Story Points completados	Bugs encontrados en testing	Bugs en producción	Lead time promedio (días)
1	30	28	3	0	4
2	32	30	5	0	4.5
3	35	25	8	1 (crítico)	7
4	34	26	7	0	6.5

Preguntas

1. **Identifica y calcula** las siguientes métricas derivadas a partir de los datos de la tabla. Muestra el procedimiento:
 - a. Velocidad promedio del equipo (SP/sprint)
 - b. Tasa de cumplimiento por sprint (SP completados / SP planificados)
 - c. Defect Escape Rate por sprint (bugs en producción / bugs totales)
 - d. Tendencia del lead time (variación sprint a sprint)
 - e. Nombra al menos otra métrica de proyecto que **no** aparezca directamente en la tabla pero que sería relevante para este caso

2. **Interpreta la tendencia:** ¿qué indican los datos sobre la salud del equipo y del proyecto? Fundamenta con al menos 3 métricas diferentes de las que calculaste.
 3. **Diagnostica la causa raíz** del bug crítico en producción del Sprint 3 usando métricas. El documento ISIII_U1 señala que, según Pressman, *"el objetivo principal de las métricas de proceso es triple: identificar ineficiencias, mejorar predictibilidad y reducir riesgos"*. ¿Cuál de estos tres objetivos se vio comprometido y qué métrica específica habría alertado del riesgo antes del Sprint 3?
 4. **Recomienda** dos acciones concretas basadas en los datos, indicando qué métrica esperarías ver mejorar con cada una y en qué plazo razonable.
-

Parte 2 — Medición del software / Métricas de producto (40 %)

A continuación se muestra el método `ProcessPayment` del módulo de pagos, donde ocurrió el bug crítico del Sprint 3:

```
public class PaymentProcessor
{
    public string ProcessPayment(Order order, string paymentType)
    {
        if (string.IsNullOrEmpty(paymentType))
            return "Invalid payment type";

        string result = "OK";

        if (paymentType == "CreditCard")
        {
            if (order.Total <= 0)
                return "Invalid amount";
            if (order.Total > 10000)
                result = "Requires manual review";
            else if (order.Total > 1000)
                result = "Requires supervisor approval";
            else if (order.Total > 100)
                result = "Quick approval";
            else
                result = "Direct payment";
        }
        else if (paymentType == "PayPal")
        {
            if (!order.HasPayPalToken)
                return "PayPal token required";
            result = "PayPal processed";
        }
        else if (paymentType == "Crypto")
        {
            if (order.Total > 5000)
                return "Crypto limit exceeded";
            result = "Crypto processed";
        }
    }
}
```

```

else
{
    result = "Unsupported payment type";
}

if (order.IsFraudSuspected)
{
    result = "Blocked: fraud suspicion";
}

return result;
}
}

```

Recordatorio de fórmulas (del documento ISIII_U1)

Si no recuerdas cómo calcular estas métricas, consulta el material de la unidad:

Métrica	Fórmula / Definición	Dónde encontrarlo en ISIII_U1
Complejidad Ciclomática (V(G))	$V(G) = E - N + 2P$ donde E = aristas, N = nodos, P = componentes conexos (normalmente 1). También: $V(G) = \text{número de decisiones} + 1$	Pág. 24 (fórmula y ejemplo), pág. 47 (umbral de riesgo > 10), glosario pág. 88
Líneas de Código (LOC)	$LOC = \text{Líneas totales} - \text{Comentarios} - \text{Líneas vacías}$. Solo cuenta código ejecutable.	Pág. 23 (fórmula y ejemplo), glosario pág. 89
Caminos independientes	Equivale al valor de V(G). Es el número mínimo de casos de prueba para cubrir todas las rutas del flujo de control.	Glosario pág. 88 ("Complejidad Ciclomática")

Preguntas

1. **Calcula manualmente** las siguientes métricas sobre el método `ProcessPayment`:

- Complejidad ciclomática (CC)
- Líneas de código (LOC) — solo código ejecutable, sin contar llaves ni líneas en blanco
- Número de caminos independientes

2. **Dibuja el grafo de flujo de control** del método. El grafo debe cumplir:

- **Nodos**: cada bloque secuencial de código (sin bifurcaciones internas) es un nodo
- **Aristas**: cada transferencia de control entre nodos es una arista
- **Decisiones**: cada bifurcación (if/else) debe tener 2 aristas salientes claramente marcadas
- A partir del grafo, verifica tu CC usando la fórmula: $CC = \text{aristas} - \text{nodos} + 2$

Puedes dibujarlo a mano y escanearlo, o usar cualquier herramienta de diagramación.

3. **Analiza el riesgo:** si la CC supera 10, el método se considera de alto riesgo de defectos. ¿Qué concluyes? Relaciona este resultado con el bug crítico que ocurrió en el Sprint 3.
4. **Cobertura de pruebas:** el equipo no mide cobertura. Según la teoría, el número mínimo de casos de prueba para lograr un 100 % de cobertura de ramas equivale a la CC. ¿Cuántos casos de prueba mínimos se necesitarían para **ProcessPayment**? Diseña uno de esos casos (entrada + resultado esperado).
5. **Propón una refactorización** del método que reduzca la CC a 8 o menos sin alterar la funcionalidad. Muestra el código refactorizado completo.

Parte 3 — Integración y reflexión (20 %)

Escribe un breve informe ejecutivo (máximo una carilla) dirigido al CTO donde:

- Resumas los hallazgos principales de la Parte 1 y 2.
- Expliques cómo las métricas de **producto** (Parte 2) explican los problemas de **proyecto** (Parte 1).
- Propongas **3 métricas clave** a monitorear en el próximo sprint, justificando por qué cada una es crítica para el caso de ShopFast.

Rúbrica de evaluación

Criterio	Peso	Excelente (100 %)	Adecuado (70 %)	Insuficiente (< 50 %)
Cálculo de métricas derivadas (Parte 1, preguntas 1-2)	15 %	Calcula correctamente las 4 métricas + identifica una adicional relevante	Calcula 3 métricas correctamente, 1 con error menor	Calcula ≤ 2 métricas o errores graves
Diagnóstico y recomendaciones (Parte 1, preguntas 3-4)	15 %	Diagnóstico preciso, relaciona métricas con el objetivo de Pressman, recomendaciones viables y con plazo	Diagnóstico parcial, relación débil con Pressman, recomendaciones genéricas	No hay diagnóstico o es incorrecto
Cálculo de CC, LOC, caminos y grafo de flujo (Parte 2, preguntas 1-2)	15 %	CC, LOC y caminos correctos. Grafo con nodos, aristas y decisiones bien marcadas. Fórmula $CC = E - N + 2$ verificada	CC o LOC correcto. Grafo incompleto o con errores menores	Cálculos incorrectos o grafo ausente

Criterio	Peso	Excelente (100 %)	Adecuado (70 %)	Insuficiente (< 50 %)
Análisis de riesgo, cobertura y refactorización (Parte 2, preguntas 3-5)	20 %	Relaciona CC > 10 con el bug, calcula casos de prueba mínimos correctamente, refactoriza reduciendo CC a ≤ 8	Relación parcial del riesgo, casos de prueba incorrectos, refactorización no reduce CC lo suficiente	No hay relación, casos ausentes, refactorización incorrecta o ausente
Informe ejecutivo (Parte 3)	20 %	Conexión clara entre métricas de producto y proyecto, 3 métricas bien justificadas y específicas para ShopFast	Conexión parcial, justificación débil, métricas genéricas	Sin conexión o métricas irrelevantes
Formato y claridad	15 %	Documento ordenado, PDF único con todo incluido, lenguaje técnico preciso	PDF completo pero con secciones desordenadas o lenguaje impreciso	No cumple con el formato solicitado o entrega ilegible

Fecha de entrega

[por definir]